

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260617_004022_245728 -->

Framework: Jest | Run ID: run_20260617_004022_245728

Tóm tắt kỹ thuật

- Framework: Jest
- Kỹ thuật test: Hybrid (kết hợp unit + integration)
- Kết quả chạy: Không có bằng chứng thực thi nào được cung cấp; do đó không thể xác định pass/fail.
- Vấn đề lớn nhất: Test code import một module `./source_under_test` không tồn tại trong thông tin nguồn được cung cấp → lỗi nghiêm trọng ngăn toàn bộ test chạy.

Lỗi nghiêm trọng

- Bằng chứng:
`const pricing = require('./source_under_test');`
Trong phần [SOURCE CODE] không có bất kỳ file nào có tên `source_under_test.js` hay bất kỳ export nào được mô tả.
- Ảnh hưởng: Khi Jest thực thi, Node sẽ ném lỗi `Cannot find module './source_under_test'`, khiến tất cả các test không thể khởi chạy được. Đây là lỗi ngăn cản hoàn toàn việc kiểm thử.
- Cách sửa:
 1. Xác định đúng file chứa các hàm `calculateDiscount`, `shippingFee`, `applyStoreCredit`.
 2. Thay đổi đường dẫn import cho phù hợp, ví dụ:
`const pricing = require('../src/pricing');` // hoặc đường dẫn thực tế
 3. Đảm bảo file được export các hàm cần thiết (`module.exports = { calculateDiscount, shippingFee, applyStoreCredit };`).

Test còn thiếu

- Không có bằng chứng nào về yêu cầu chi tiết, kế hoạch test, hoặc source code, nên không thể khẳng định các trường hợp biên, ngoại lệ, bảo mật đã được bao phủ.
- Cách sửa:
 - Thêm các test cho:
 - Giá trị discount khi `amount < 0` hoặc tier không hợp lệ (ngoại trừ case đã có).
 - Trường hợp `shippingFee` với trọng lượng = 0, hoặc `isFragile = false`, hoặc `destination` không phải `'domestic'/'international'`.
 - Kiểm tra `applyStoreCredit` khi `storeCredit > orderTotal` và khi `storeCredit` hoặc `orderTotal` là số âm.

Assertion yếu

- Các assertion hiện tại đều sử dụng `toBe` hoặc `toThrow` với giá trị cụ thể, không có dấu hiệu luôn đúng hay chỉ in log.
- Tuy nhiên, vì không có bằng chứng về giá trị thực tế của các hàm trong source, các giá trị kỳ vọng (50, 22.5, 0) không thể xác nhận là đúng.
- Cách sửa:

- Khi source code được cung cấp, xác nhận lại các giá trị kỳ vọng dựa trên logic thực tế.
- Nếu các giá trị phụ thuộc vào tính toán động (ví dụ: phí vận chuyển dựa trên tỷ lệ), cân nhắc dùng `toBeCloseTo` với độ chính xác phù hợp.

Rủi ro maintainability

- Rủi ro: Việc import một module không tồn tại khiến bộ test dễ “break” khi cấu trúc thư mục thay đổi.
- Cách giảm rủi ro:
 - Sử dụng đường dẫn tương đối rõ ràng và đồng nhất với cấu trúc dự án.
 - Thêm comment mô tả vị trí file nguồn.
 - Đặt các test trong cùng thư mục hoặc cấu trúc tests để Jest tự động resolve.

Gợi ý sửa ngay

1. Sửa import: Xác định và sửa đường dẫn tới file thực tế chứa các hàm cần test.
2. Thêm test biên: Viết ít nhất một test cho mỗi hàm kiểm tra các giá trị đầu vào biên (0, âm, giá trị lớn).
3. Cập nhật giá trị kỳ vọng: Khi source code được cung cấp, rà soát lại các giá trị `toBe` để chắc chắn chúng phản ánh đúng logic.
4. Cung cấp execution evidence: Chạy Jest và đính kèm log kết quả (`npm test -- --detectOpenHandles`) để xác nhận các test thực sự pass/fail.

Nếu không có source code và bằng chứng thực thi, không thể đánh giá đầy đủ tính đúng đắn của các test hiện có; việc cung cấp các tài liệu này là bắt buộc để hoàn thiện quá trình review.